



INDIA.RUNS

Build what next India runs on

Team Name : **Innovate Bharat**

Team Leader Name : **Manogna Shanigarapu**

Problem Statement : **AI-powered candidate ranking system that evaluates job fit beyond keyword matching**

Solution Overview

What is your proposed solution?

A four-stage retrieval-and-ranking pipeline that reads the job description, builds a semantic profile of the ideal candidate, and scores all 100,000 candidates against it using a hybrid of embedding similarity, verified-skill matching, career-trajectory analysis, and behavioral signals -- producing a ranked Top-100 shortlist with grounded, fact-based reasoning for each candidate.

What differentiates this from traditional candidate matching?

- Cross-checks claimed skills against Redrob's own measured skill_assessment_scores, directly defeating keyword-stuffing rather than rewarding it.
- Rule-based honeypot detection (logical-impossibility checks) excludes fabricated profiles before they ever reach scoring.
- Behavioral availability (activity, response rate) is a multiplicative modifier, not a hard filter -- a strong-fit, quiet candidate still outranks a weak-fit, active one.
- Every Top-100 reasoning string is built from verified profile facts, not generated by an LLM -- hallucination is structurally impossible.

JD Understanding & Candidate Evaluation

Key requirements extracted from the JD

- 5-9 years experience (soft band, not a hard cutoff).
- Production embeddings-based retrieval (sentence-transformers, BGE/E5).
- Vector database / hybrid search (Pinecone, Weaviate, Qdrant, FAISS).
- Rigorous evaluation background (NDCG, MRR, MAP, A/B testing).
- Disqualifiers: consulting-only career, pure research, job-hopping, non-coding seniority, "LangChain-tourist" AI.

Candidate signals used beyond keyword matching

- Career narrative compared semantically against the JD's ideal-candidate profile, not just titles or skill lists.
- Claimed skill proficiency cross-checked against Redrob's independently measured assessment scores.
- Career trajectory patterns (consulting-only, job-hopping, non-coding seniority) and platform activity (recency, response rate).

Ranking Methodology

How the system retrieves, scores, and ranks candidates

- Stage A (structural gate): rule-based exclusion of honeypots and out-of-scope candidates across the full 100,000-candidate pool.
- Stage B (semantic retrieval): cosine similarity between a JD-narrative embedding and each candidate's career narrative.
- Stage C (hybrid scoring): combines 7 weighted components into one score, then applies a behavioral-availability modifier.
- Stage D (reasoning): generates a short, fact-grounded justification for each of the Top 100.

Models, algorithms, and heuristics used

- sentence-transformers (all-MiniLM-L6-v2) bi-encoder for semantic similarity, with an automatic TF-IDF + cosine-similarity fallback.
- Weighted linear composite: semantic fit 0.32, skills match 0.28, title relevance 0.16, experience fit 0.08, location fit 0.08, notice period 0.04, career trajectory 0.04.
- $\text{final_score} = \text{base_score} \times \text{behavioral_modifier}$, where the modifier is strictly multiplicative, range [0.50, 1.10].

Explainability & Data Validation

How ranking decisions are explained

- Each Top-100 candidate gets a short, fact-based justification built from priority-ordered clauses: role fit, the strongest concern (if any), skill-match evidence, logistics, and behavioral signal.

Preventing hallucination or unsupported justification

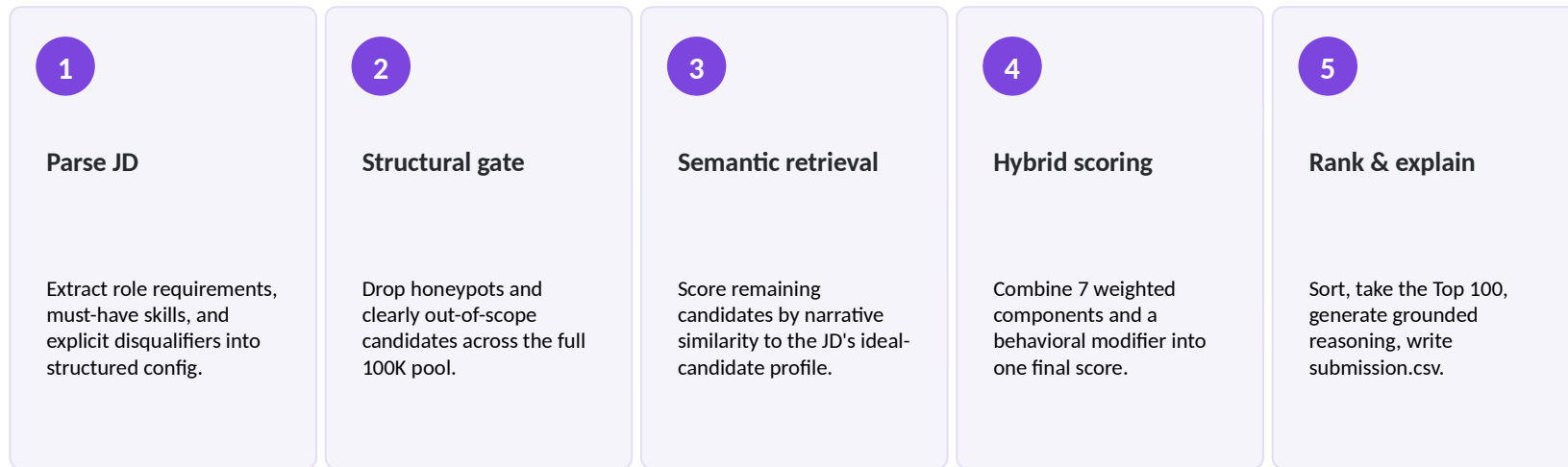
- Reasoning is template-built, not LLM-generated -- a clause is only emitted when its underlying structured signal is independently verified true in the candidate's actual profile data.

Handling inconsistent, low-quality, or suspicious profiles

- Honeypot detection via three logical-impossibility checks: "expert" proficiency claimed with zero months of use, 8+ simultaneous expert-rated skills, and career history that predates what stated experience implies.
- Skill-match scoring discounts claims that contradict Redrob's own measured skill_assessment_scores, directly catching keyword-stuffed profiles.

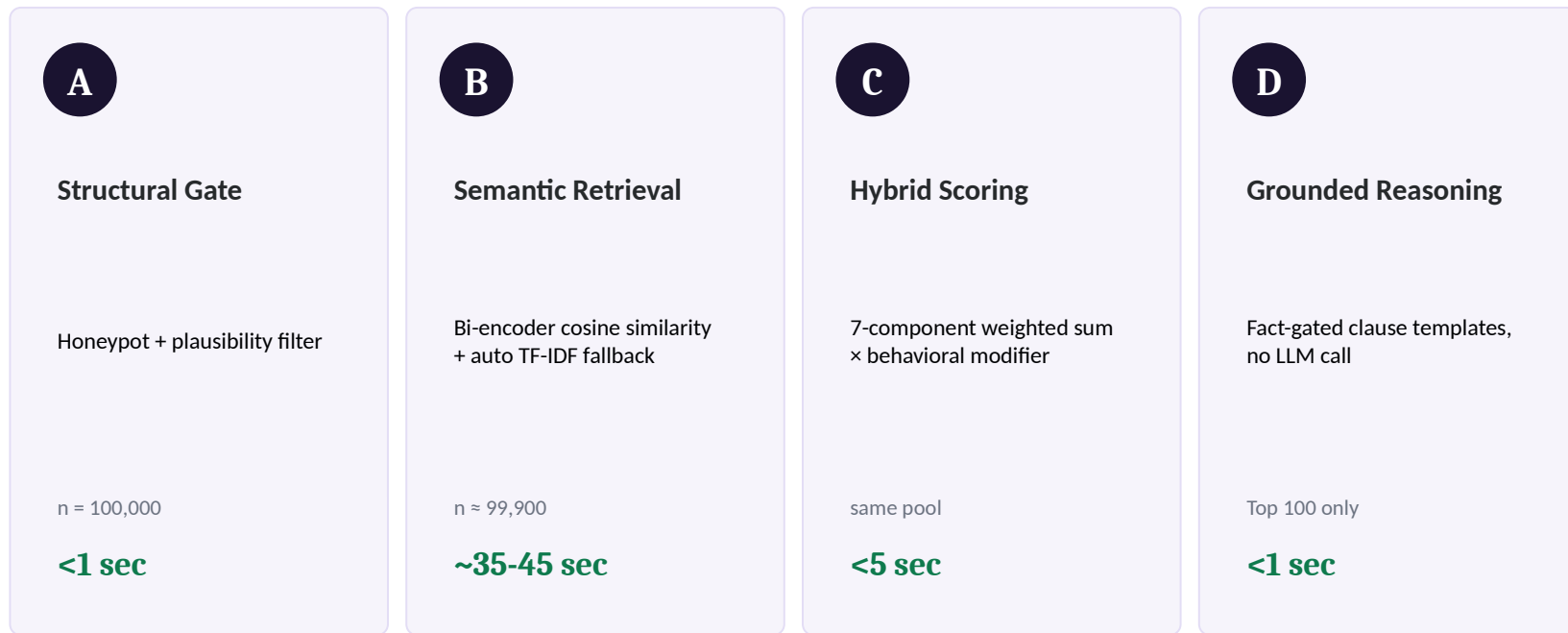
End-to-End Workflow

Complete workflow: JD input to ranked candidate output



Measured end-to-end: ~51 seconds against the real 100,000-candidate dataset, on a single CPU core (5-minute budget).

System Architecture



Constraint: ≤5 min wall-clock, ≤16GB RAM, CPU-only, no GPU, no network, n=100,000. Per-candidate LLM inference is ~33x over budget at 100ms/call -- the funnel narrows the pool before any expensive computation runs.

Results & Performance

Ranking quality (validated against the real 100,000-candidate dataset)



Official validator

"Submission is valid."



Consulting-only / research-only leakage

0 candidates each



Tier 2-3 relevant titles in Top 100

99 / 100



Honeypot rate in Top 100

0.0% (0 / 100)



LangChain-tourist leakage

0 candidates



Unique reasoning strings

100 / 100

Compute / runtime compliance

- Total measured runtime: 50.7 seconds on a single CPU core, against a 5-minute budget.
- Peak memory and disk usage remain well within the 16GB RAM / 5GB intermediate-state limits -- no GPU, no network access required during the timed run.

Technologies Used

What technologies, frameworks, and tools were used -- and why

- Python -- the entire pipeline (rank.py, src/redrob_ranker/) for portability and clarity in a CPU-only, no-network sandbox.
- pandas / numpy -- vectorized feature extraction and scoring across the full 100,000-candidate pool within the time budget.
- sentence-transformers (all-MiniLM-L6-v2) -- small, CPU-feasible bi-encoder for semantic similarity, chosen for its quality/footprint balance (~80MB).
- Streamlit -- hosted sandbox demo (Streamlit Community Cloud) for live, interactive reproduction.
- scikit-learn (TfidfVectorizer + cosine_similarity) -- zero-extra-dependency fallback, auto-selected if sentence-transformers is unavailable.
- Python's unittest -- 27 tests including a real-data regression test against 5 actual honeypot candidates, zero added dependency risk.

Submission Assets

GitHub repository

<https://github.com/manogna2502/redrob-candidate-ranker>

Live sandbox demo

<https://redrob-candidate-ranker-teaminnovatebharat.streamlit.app>

Ranked output file

submission.csv -- Top 100 ranked candidates, included in the GitHub repository root.



INDIA.RUNS

Build what next India runs on

THANK YOU